
Artificial Intelligence

BS (CS) Spring 2026

Lab 15 Manual



Learning Objectives:

1. Neural Networks

Lab Manual

1. Neural Network Theory: The Digital Brain

Neural Networks are a subset of machine learning designed to mimic the way biological neurons in the human brain process information. They are the foundation of Deep Learning, allowing computers to learn from unstructured data like images and text.

The Artificial Neuron (Perceptron)

The basic building block is the neuron. It takes multiple inputs, multiplies them by Weights (importance), adds a Bias (threshold), and passes the result through an Activation Function.

Key Theoretical Concepts

- **Weights and Biases:** Weights determine the strength of the connection between neurons, while the bias allows the activation function to be shifted to better fit the data.
- **Activation Functions:** These introduce non-linearity, allowing the network to solve complex problems that aren't just straight lines. Common types include:
 - **ReLU (Rectified Linear Unit):** Most popular; outputs the input if positive, otherwise zero.
 - **Sigmoid/Logistic:** Squashes values between 0 and 1, useful for probability.
- **Forward Propagation:** The process where input data travels through the layers to generate an output.
- **Backpropagation:** The "learning" phase. The model calculates the error (Loss) and goes backward to adjust the weights to reduce that error in the next round.

2. Network Architecture

A typical network is organized into a hierarchy of layers:

- **Input Layer:** The entry point for features (e.g., pixel values or data columns).
- **Hidden Layers:** Layers between input and output where complex patterns are extracted. A network with many hidden layers is called a "Deep" network.
- **Output Layer:** The final layer that produces the prediction, such as a category or a numerical value.

3. Environment Setup

Install the core libraries for processing and visualization:

```
pip install numpy scikit-learn matplotlib seaborn
```

Standard Imports:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

4. Implementation (MLP Classifier)

We will use a Multi-Layer Perceptron (MLP) to solve a classification task.

```
from sklearn.datasets import make_circles

# 1. Generate non-linear data
X, y = make_circles(n_samples=500, noise=0.1, factor=0.5,
                    random_state=42)

# 2. Scaling (CRITICAL for Neural Networks)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Training the model
# hidden_layer_sizes=(8, 4) creates two layers with 8 and 4 neurons
# respectively
mlp = MLPClassifier(hidden_layer_sizes=(8, 4), activation='relu',
                    max_iter=1000, random_state=42)
mlp.fit(X_scaled, y)

print(f"Training Accuracy: {mlp.score(X_scaled, y):.2f}")
```

5. Troubleshooting & Best Practices

Issue	Theoretical Reason	Solution
Slow Convergence	Weights are not updating efficiently.	Ensure data is scaled using StandardScaler.
Overfitting	The model is "memorizing" noise in the training data.	Increase the alpha (regularization) parameter.
Flat Loss Curve	The learning rate is too small or weights are stuck.	Change the solver to 'adam' or increase the learning rate.